



PROJECT REPORT

HIAS

Hall Invigilator Allocation System

St. Xavier's Catholic College of Engineering (Autonomous), Nagercoil

Developed by **Anushath S, Akhil Koska A** and **Harish S**
Version 1.0.0 · Windows desktop application

Contents

1	Abstract
2	Problem statement and objectives
3	Features
4	System architecture
5	Technology stack
6	Database design
7	Allocation algorithm and validation rules
8	Testing and quality
9	Screenshots
10	Limitations and future work
11	Conclusion

1. Abstract

Allocating invigilators to examination halls is a recurring, manual task in every college. Done by hand, it is slow and error-prone: the same person can be sent to the same hall repeatedly while others are not, a hall can be double-booked, and there is no record to check fairness against afterwards.

HIAS (Hall Invigilator Allocation System) is a Windows desktop application that automates this task. For every exam session it assigns each on-duty staff member to a hall using a **rotation algorithm** that remembers each person's previous halls, validates the result against seven rules, allows controlled manual edits with an audit trail, and produces printable PDF allocation sheets and reports. It runs completely offline with an embedded SQLite database, so it can be installed on any college computer without a server or internet connection.

2. Problem statement and objectives

Problem

- Manual allocation for each session takes significant time for the examination cell.
- Fair rotation is hard to guarantee by memory or spreadsheets: staff may repeat halls before others have had a turn.
- Mistakes such as one hall assigned to two people, or one person to two halls, are easy to make and are found late.
- There is no reliable history to answer "Who was in hall H104 on 3 November?" or to justify an allocation.

Objectives

1. Generate a complete, fair hall allocation for a session in one click.
2. Rotate halls across sessions and exam series so nobody repeats a hall before completing the rotation.
3. Validate every allocation automatically before it is finalised.
4. Allow justified manual changes while keeping a record of what was changed and why.
5. Produce printable, college-branded reports.
6. Work offline on an ordinary Windows computer with simple installation and backup.

3. Features

Master data

Departments, halls (code, block, floor, capacity, active flag) and staff, including bulk import from Excel with row-by-row error reporting.

Allocation batches

A three-step wizard creates an exam series from calendar dates, flags Sundays and public holidays, and sets FN/AN sessions with timings.

One-click allocation

Choose the staff and halls for a session by ticking, by searching name or Staff ID, or by uploading an Excel duty list; the rotation engine assigns everyone a hall.

Rule validation

Seven rules (R1–R7) are checked before confirmation; violations are explained by name.

Controlled edits

Manual changes offer only rule-safe halls and are recorded as "Admin Edited" with an optional reason.

Reports and PDF

Session allocation sheet, staff-wise duties, date-wise hall sheet, complete timetable grid and rotation audit, all exportable to PDF with the college letterhead.

History and audit

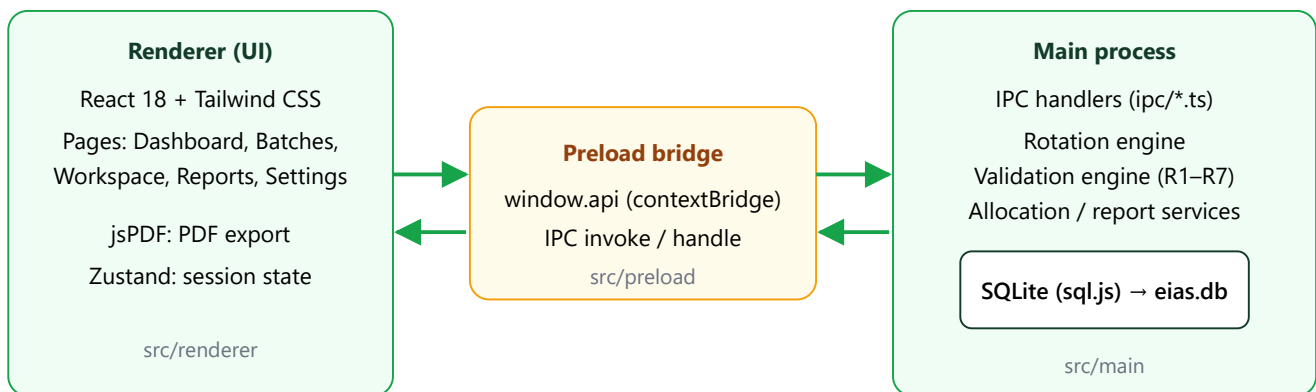
Search past allocations by batch, session, staff or hall; key actions are written to an audit log.

Offline and safe

No internet or server needed; one-click backup and restore; destructive actions require typed confirmation.

4. System architecture

HIAS is an **Electron** application with three layers. The user interface never touches the database directly; every operation goes through a narrow, typed IPC bridge to the main process, which owns the business logic and the database file.



- **Security:** the UI runs with `contextIsolation` on and Node integration off; it can only call the functions the preload bridge exposes.
- **Data safety:** multi-step operations (confirming an allocation, deleting a batch) run inside database transactions, and the database file is written to disk after every committed change.
- **Offline:** fonts and all assets are bundled; the packaged app makes no network requests (verified during testing).
- **Web build:** the same UI can also be built as a browser application (`npm run build:web`) that stores data in the browser; the desktop application is the supported deployment.

5. Technology stack

Area	Technology
Desktop shell	Electron 32, packaged as a Windows NSIS installer with electron-builder
Build tooling	electron-vite (Vite 5), TypeScript 5 (strict mode)
User interface	React 18, React Router 6, Tailwind CSS 3, Lucide icons, bundled Inter font
State	Zustand
Database	SQLite via sql.js (WebAssembly; no native compilation needed)
Reports	jsPDF and jspdf-autotable (PDF), SheetJS xlsx (Excel import)
Security	bcryptjs password hashing (web build), Electron context isolation
Testing	Vitest (unit and integration), scripted end-to-end run of the packaged app
Version control	Git and GitHub, collaborative development through pull requests

6. Database design

The database is a single SQLite file (%APPDATA%\eias\eias.db) with foreign keys and cascading deletes. Schema changes are applied through versioned migrations at start-up.

Table	Purpose	Key columns
departments	Academic departments	code (unique), name
users	Staff/invigilators and the administrator	staff_id (unique), name, department_id, role, is_active
halls	Examination halls	hall_code (unique), block, floor, capacity, sort_order, is_active
exam_cycles	Allocation batches (exam series)	name, academic_year, status
exam_sessions	One FN/AN session on a date	cycle_id, exam_date, session_type, rotation_step, timings, status
allocations	Staff-to-hall assignments per session	session_id, user_id, hall_id, generated_hall_id, is_manually_edited, edit_reason; unique per (session, user) and (session, hall)
rotation_history	Confirmed assignments over time; the memory used by the rotation	user_id, session_id, hall_id, global_order
settings	College profile and default timings	key, value
audit_log	Record of key actions (edits, confirmations, rotation restarts, deletions)	action, description, payload, created_at
notifications	In-app notices for staff (web build)	user_id, title, message, is_read

7. Allocation algorithm and validation rules

Rotation algorithm

1. The halls selected for a session form its **hall pool**.
2. For each selected staff member, HIAS reads their confirmed hall history from `rotation_history`.
3. From that history HIAS works out each person's **current rotation cycle**: replaying their visits from oldest to newest, every hall of the pool they visit is crossed off, and once every hall of the pool has been crossed off the cycle is complete and starts again. Halls not yet crossed off are the ones still open to them.
4. Every (staff, hall) pair gets a cost made of four parts: a very large penalty if the hall is already crossed off in the person's current cycle (rule R1); how often they have had that hall in total, so less-visited halls are preferred; a small penalty if it is the hall they had last time, so nobody gets the same hall twice in a row when a new cycle starts; and a **random draw** between 0 and 1, which is smaller than every other part and so only decides between otherwise equal choices.
5. All staff of the session are assigned **together** by finding the minimum-cost perfect matching (the **Hungarian algorithm**, $O(n^3)$). Because the R1 penalty outweighs everything else, no rule is broken whenever a valid assignment exists. Which of their open halls each person gets is decided by the random draw, so assignments do not follow a fixed "next hall" sequence, and every cycle is drawn afresh.
6. **Seeded draw**. The random draw is seeded from the batch, the session (its id and creation time) and the selected staff and halls. Regenerating the same draft without changing the selection gives exactly the

same result, whatever order the staff and halls were ticked in; a different session, batch or selection gives an independent order.

7. **Unavoidable repeats.** When the staff on duty vary, a repeat can be mathematically unavoidable (for example, two people whose only open hall is the same one). The algorithm then uses the minimum possible number of repeats, and the repeated hall is drawn at random from the halls the person has already had in the current cycle, avoiding the one they had most recently. A final check refuses any result that double-books a hall or a person, or has more repeats than the minimum.
8. The result is saved as a **draft**. Only when the administrator **confirms** it is it written to `rotation_history`, inside a transaction, which moves the rotation forward.

Because the rotation history is global rather than per batch, fairness carries over from one exam series to the next. An administrator can reset it deliberately (Restart Rotation), for example at the start of an academic year.

Validation rules

Rule	Check
R1	No repeated hall within a cycle: a staff member may not return to a hall of the session's pool until they have had every hall of that pool once; the cycle then resets and every hall is open again. Halls outside the session's pool are ignored.
R2	A hall has at most one invigilator in a session.
R3	A staff member has at most one hall in a session.
R4	Rotation integrity: a session follows the last confirmed session in step order.
R5	Staff must be active.
R6	Halls must be active.
R7	The number of staff equals the number of halls.

R1 is enforced strictly with one exception: if an allocation has no more repeats than the proven minimum for the staff and halls of that session, the repeats are shown to the administrator as **warnings** ("unavoidable repeat") instead of blocking the session. Manual edits are checked against the rules as they are made, and the edit dialog only offers halls that keep the allocation valid.

Why a matching algorithm

An earlier version assigned halls one person at a time. Fairness testing showed that this worked perfectly for a fixed team but failed badly when more staff than halls take turns (the normal case in a college): two people competed for the same "next" hall and one was pushed into a recently used hall, so the system rejected its own allocation. With 10 staff sharing 6 halls, only 5 of 60 sessions could be confirmed; with a random duty list of 6 out of 12 staff, 9 of 80. Assigning everyone together as an optimal matching fixed this: all sessions in every tested scenario now confirm.

8. Testing and quality

Automated tests

101 automated tests (Vitest) run against an in-memory database and all pass. They cover:

- The rotation algorithm: rotation cycles (no hall repeats until every hall of the pool has been visited, then the cycle resets), the seeded random draw (the same selection gives the same draft; different sessions give different orders), staff who join later, pools smaller than the total number of halls, and reading history in the correct order across batches.
- Every validation rule, including a regression test for a full-cycle restart that an earlier version wrongly blocked.
- The allocation lifecycle: generate, edit, confirm, delete draft sessions, permanent staff deletion, rotation restart, and batch status updates.
- A **fairness suite** of 29 multi-session scenarios run through the real engine, validator and database (below).

Fairness testing

Each scenario generates and confirms every session in turn and checks that no hall or person is double-booked. "Hall spread" is, for the worst-off person, the difference between the hall they had most often and least often.

Scenario	Sessions confirmed	Unavoidable repeats	Same hall twice in a row	Hall spread
Fixed team: 6 staff, 6 halls (also 2, 3, 5 and 9)	36 / 36	0	0	0 (every hall exactly 6 times)
10 staff taking turns in 6 halls	60 / 60	4–11 of 360 duties (limit 4.6%)	0–1	0–2 (limit 3)
Random duty list: 6 of 12 staff each session	120 / 120	10 of 720 duties	1	1
Random duty list: 8 of 15 staff (3 runs)	240 / 240	19 of 1,920	0	1
New staff join half-way	24 / 24	0	0	0
A hall closes for 10 sessions and reopens	28 / 28	10	0	2
Manual swap by the administrator	12 / 12	0	0	0
Rotation continues across two batches	12 / 12	0	0	0
Restart rotation part-way	16 / 16	0	0	0
Same team morning and afternoon every day	20 / 20	0	0	0
Large college: 30 of 60 staff, 30 halls	60 / 60	0–4 of 1,800	0	1–2 (6–22 ms per session)

With 10 staff taking turns in 6 halls, a few repeats are unavoidable. This is inherent to the staff/hall ratio, not a defect: each person's cycle is drawn independently, so the halls the six people on duty still need do not always line up into a repeat-free assignment. The engine then makes the minimum number of repeats and warns the administrator. Over 20 independent draws the worst case was 11 of 360 duties (3.1%); the test allows up to 4.6%, and a hall spread of up to 3, the same limit used for random duty lists.

A second set of 17 scenarios covers messier situations, edge cases and attempts to break the rules. All pass:

Scenario	Result
A different number of halls each day (3 to 8)	60 / 60 confirmed; hall counts within 2 on the halls open every day
A different set of halls each session (5 of 8)	80 / 80 confirmed in every run; 12–30 unavoidable repeats in 400 duties; hall spread 2–4
A hall added permanently; a teacher on long leave returns	All confirmed; newcomers and returners slot in within 1–3
A single hall	All confirmed, no false warnings
Regenerating a draft several times	Identical result every time; nothing recorded until confirmed
Ticking staff and halls in a different order	Same person gets the same hall
Confirming sessions out of order; mismatched counts; inactive staff or hall	Refused with R4, R7, R5 and R6
History built so that a repeat is forced	Exactly one repeat (the minimum), drawn at random from the halls already used in the person's current cycle, avoiding their most recent hall; confirmed with a warning
Manual edit onto an occupied hall or a hall outside the session	Refused; nothing changes
Deleting a draft part-way	The deleted draft leaves nothing in the rotation history; the next session is valid, and after 10 sessions everyone has had every hall exactly twice
300 sessions with a fixed team	Every hall exactly 50 times each, no repeats
Two batches running side by side; 20 newcomers starting together	No repeats; everyone has every hall exactly 4 times (batches) or twice (newcomers)
Very large college: 100 of 150 staff, 100 halls	All confirmed, about 0.1 s per session

Figures given as a range (10 staff taking turns, large college, a different set of halls each session) are the lowest and highest values over 20 independent runs, each with a different random draw and, where the duty list is random, a different duty list. All other figures are from a single run with a fixed seed, not a measured worst case.

With a stable team, every person has every hall exactly once per cycle, in a randomly drawn order, with no repeats. When staff take turns or the duty list is random, every session still confirms, but a few repeats can be unavoidable; they are kept to the proven minimum and each person's hall counts stay within 1–4 of each other. How many duties each person gets depends on who the administrator puts on duty, which the engine does not choose.

TypeScript runs in strict mode and the whole code base type-checks without errors (`npm run typecheck`).

End-to-end verification

The packaged Windows application was driven automatically against a separate test data folder with sample data (3 departments, 6 halls, 8 staff, one batch of 4 sessions). The run checked:

This run predates the change to the random-draw rotation algorithm (Section 7); read it as a historical record of the packaged-app workflow, not as a measurement of current allocation behaviour.

Check	Result
Generate and confirm four sessions	Passed, 0 rule violations
30 sessions with 10 staff taking turns in 6 halls; 20 sessions with random duty lists	All 50 confirmed
Batch status moves from Draft to Confirmed	Passed
Excel import with two valid and two invalid rows	2 imported, 2 reported with reasons
Delete a batch; database backup	Passed
All screens load without errors	Passed
Session PDF and all four report PDFs	All valid PDFs, content checked
Network access while running	None (fully offline)

This run found and fixed several defects before release, including an allocation engine that rejected its own allocations when staff took turns, empty staff-wise reports, a dashboard counter that was never calculated, batches that never left the Draft state, the order of sessions in the timetable, and a restore that could be undone by further use before a restart.

9. Screenshots

MAIN

Dashboard

Allocation Batches

Allocation History

Reports

MASTER DATA

Staff / Invigilators

Halls

Departments

SYSTEM

Settings

← Back

November 2026 End Semester Exams

2026-27 · Allocation Workspace

02 Nov 2026 FN

02 Nov 2026 AN

03 Nov 2026 FN

04 Nov 2026 FN

+ Add Session

Date: Mon, 02 Nov 2026

Session: Forenoon (FN)

Report: 09:30 AM

Exam: 10:00 AM – 01:00 PM

Confirmed

Export PDF

Edit Schedule & Timings

#	STAFF ID	NAME	DEPARTMENT	ASSIGNED HALL	TYPE	
1	STF001	Anitha R	CSE	H101	Auto-generated	
2	STF004	Dinesh K	CSE	H104	Auto-generated	
3	STF002	Bala S	ECE	H102	Auto-generated	
4	STF005	Esther J	ECE	H105	Auto-generated	
5	STF003	Chitra M	MECH	H103	Auto-generated	
6	STF006	Faizal A	MECH	H106	Auto-generated	

6 invigilators · 0 admin-edited

Allocation workspace: a confirmed session with its generated hall assignments.

MAIN

Dashboard

Allocation Batches

Allocation History

Reports

MASTER DATA

Staff / Invigilators

Halls

Departments

SYSTEM

Settings

← Back

Reports

Generate and export allocation reports with your college letterhead

Staff-Wise Duty Report

Date-Wise Allocation Sheet

Complete Timetable Grid

Rotation Audit Report

Allocation Batch *

November 2026 End Semester

Generate

Export PDF

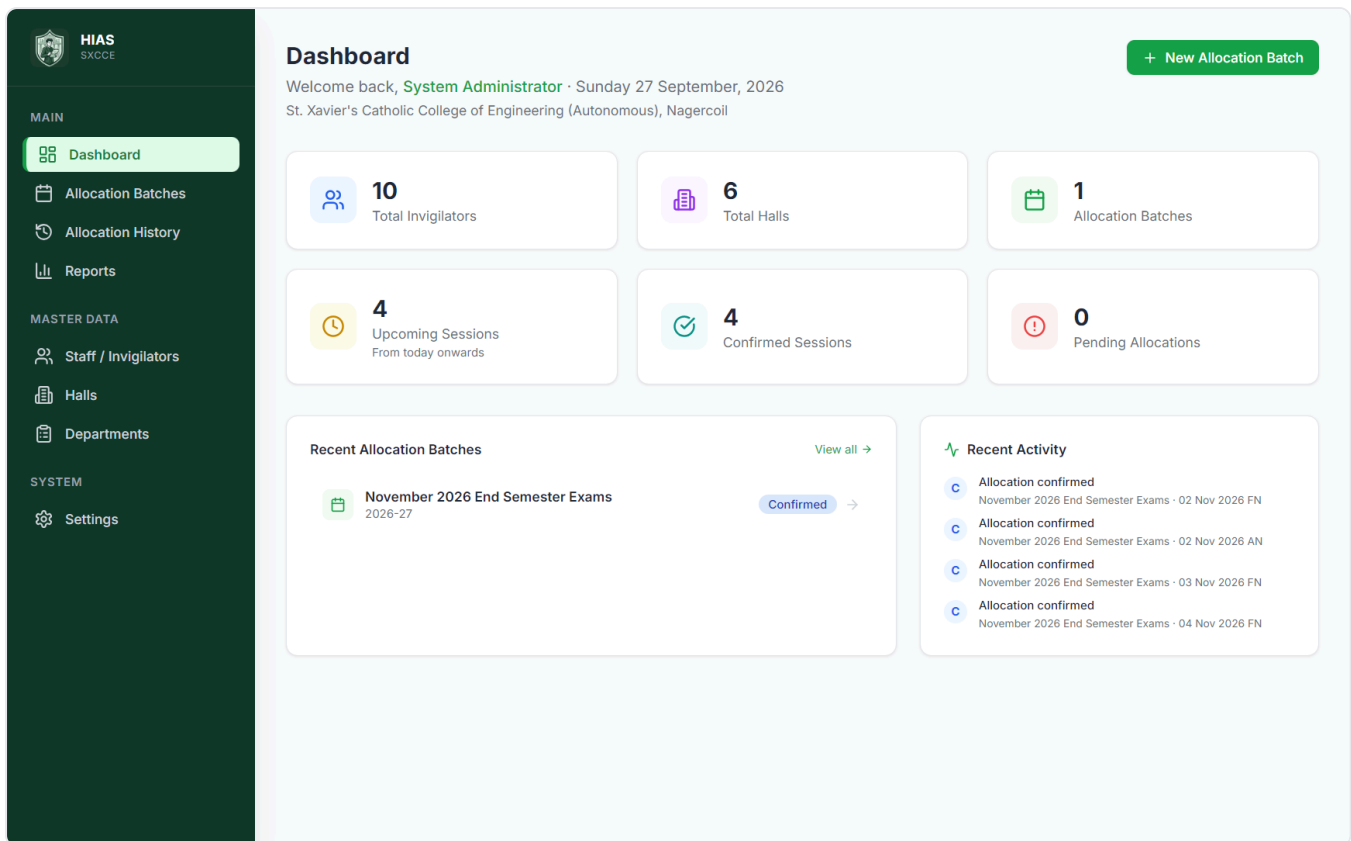
Complete Timetable Matrix — November 2026 End Semester Exams

Staff	02 Nov FN	02 Nov AN	03 Nov FN	04 Nov FN
Anitha R STF001	H101	H102	H103	H104
Dinesh K STF004	H104	H105	H106	H101
Gokul P STF007	—	—	—	—
Imran K STF101	—	—	—	—
Bala S STF002	H102	H103	H104	H105
Esther J STF005	H105	H106	H101	H102
Hema V STF008	—	—	—	—
Janani L STF102	—	—	—	—

Complete timetable grid: every staff member against every session of a batch.

HIAS – Hall Invigilator Allocation System

Page 12 of 14



Dashboard.

All names shown are sample data used for testing.

10. Limitations and future work

Current limitations

- **Single computer.** Data lives on the computer where HIAS is installed; several computers cannot share it live. Backup and restore move data between computers.
- **No user accounts in the desktop app.** Anyone who can open the app has full access, so it should be installed on a computer used only by the examination cell.
- **Unsigned installer.** Windows shows a SmartScreen warning on first install.
- **Hall order** is fixed by creation order; there is no screen to reorder halls.

Possible enhancements

- A shared network database so several computers can work on the same data.
- Optional administrator password for the desktop app.
- Excel export of reports, and SMS or e-mail duty notifications to staff.
- Workload balancing so the number of duties per person evens out across a batch.
- Hall reordering and seat-capacity-based allocation.

11. Conclusion

HIAS replaces a slow, manual and error-prone process with a reliable tool: an entire session is allocated in one click, fairness is enforced by a rotation that remembers every confirmed duty, mistakes are caught by automatic rule checks before anything is finalised, and every change is recorded. Delivered as an offline Windows application with printable reports, backup and a user guide, it is ready to be used by the examination cell.